Soubor: City.java

```java
import java.util.Random;

public class City {
    private double temp;
    private String name;

    Random rand = new Random();

    public City(String name, double temp) {
        this.name = name;
        this.temp = temp;
    }

    public double getTemp() {
        return temp;
    }

    public String getName() {
        return name;
    }

    public String stringExtensionCity() {
        return this.name + ": ";
    }
}

```

Soubor: Rows.java

```java
import java.io.*;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.StringJoiner;

public class Rows {

    HashMap<String, ArrayList<Double>> data = new HashMap<>();
    HashMap<String, ArrayList<Double>> dataAverage = new HashMap<>();


    public void gatherInformation() throws Exception {
        BufferedReader bufferedReader = new BufferedReader(new FileReader("maturita/rows/data.txt"));

        while (bufferedReader.ready()) {
            String line = bufferedReader.readLine();
            String[] lineData = line.split(";");
            String city = lineData[0];
            String temperature = lineData[1].replace(",", ".");

            data.putIfAbsent(city, new ArrayList<>());
            data.get(city).add(Double.parseDouble(temperature));
        }

        bufferedReader.close();
    }

    public void printInformation() {
        for (String city : data.keySet()) {
            System.out.println(city + "; " + data.get(city));
        }
    }

    public void createFinal() {
        for (String city : data.keySet()) {
            dataAverage.putIfAbsent(city, new ArrayList<>());
            dataAverage.get(city).add(getMinimumTemperature(data.get(city)));
            dataAverage.get(city).add(getAverageTemperature(data.get(city)));
            dataAverage.get(city).add(getMaximumTemperature(data.get(city)));
        }
    }

    public void printFinal() {
        StringJoiner stringJoiner = new StringJoiner(", ");

        for (String city : dataAverage.keySet()) {
            stringJoiner.add(city + "=" + dataAverage.get(city).get(0) + "/" + dataAverage.get(city).get(1)
                    + "/" + dataAverage.get(city).get(2));
        }

        System.out.println(stringJoiner.toString());
    }

    private double getAverageTemperature(ArrayList<Double> temps) {
        Double sum = 0.0;

        for (Double temp : temps) {
            sum += temp;
        }

        return Math.round((sum / temps.size()) * 10.0) / 10.0 ;
    }

    private double getMinimumTemperature(ArrayList<Double> temps) {
        return Collections.min(temps);
    }

    private double getMaximumTemperature(ArrayList<Double> temps) {
        return Collections.max(temps);
    }



```

Soubor: Rows.java

```
75      public static void main(String[] args) throws Exception {
76
77          Rows demo = new Rows();
78
79          demo.gatherInformation();
80          demo.printInformation();
81
82          demo.createFinal();
83          demo.printFinal();
84      }
85  }
86
```

Soubor: Cross.java

```java
package backTrackAlgorithms;

public class Cross {
    private final int[] cr;
    private int level;

    public Cross() {
        this.cr = new int[5];
        level = -1;
        backTrack();
    }

    public int[] getCr() {
        return cr;
    }

    public void backTrack() {
        level++;

        for (int i = 1; i <= 8; i++) {
            cr[level] = i;
            if (test()) {
                backTrack();
            }
        }

        level--;
    }

    public boolean test(){
        return switch (level) {
            case 1 -> cr[0] + cr[1] == 10;
            case 2 -> cr[0] > cr[2];
            case 3 -> cr[0] - cr[3] > 2;
            case 4 -> {
                if (cr[0] * cr[4] > 20) {
                    System.out.printf("  %d  \n", cr[1]);
                    System.out.printf("%d %d %d\n", cr[2], cr[0], cr[3]);
                    System.out.printf("  %d  \n", cr[4]);
                    System.out.println();
                }
                yield false;
            }
            default -> true;
        };

    }

    public int getLevel() {
        return level;
    }

    public void setLevel(int level) {
        this.level = level;
    }
}

```

Soubor: GCD_LCM.java

```java
public class GCD_LCM {
    private final long num1;
    private final long num2;

    public GCD_LCM(long num1, long num2) {
        if (num1 == num2){
            this.num1 = num1;
            this.num2 = num2;
        }
        else {
            this.num1 = Math.max(num1, num2);
            this.num2 = Math.min(num1, num2);
        }
    }

    public long getNum1() {
        return num1;
    }

    public long getNum2() {
        return num2;
    }

    private long GCD(long num1, long num2) {
        if (num2 != 0) {
            return GCD(num2, num1 % num2);
        }
        else {
            return num1;
        }
    }

    public long getGCD(){
        return GCD(num1,num2);
    }

    public long getLCM(){
        return (num1*num2)/(GCD(num1,num2));
    }

    public static void main(String[] args) {
        GCD_LCM gcdLcm = new GCD_LCM(4, 7);
        System.out.printf("GCD(%d, %d) = %d\n", gcdLcm.getNum1(), gcdLcm.getNum2(), gcdLcm.getGCD());
        System.out.printf("LCM(%d, %d) = %d\n", gcdLcm.getNum1(), gcdLcm.getNum2(), gcdLcm.getLCM());
    }
}

```

Soubor: RowsGen.java

```java
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.Random;

public class RowsGen {

    BufferedWriter bufferedWriter;

    ArrayList<City> cities;

    Random rand = new Random();

    int numOfLines;

    public RowsGen(String filePath, ArrayList<City> cities, int numOfLines) throws IOException {
        bufferedWriter = new BufferedWriter(new FileWriter(filePath, false));
        this.cities = cities;
        this.numOfLines = numOfLines;

        long time = System.currentTimeMillis();
        generateText();
        System.out.printf("Time taken: %d", System.currentTimeMillis() - time);
    }

    public void generateText() throws IOException {
        for (int i = 0; i < numOfLines; i++) {
            writeText(getRandomCity(), getRandTempDifference());
        }

        bufferedWriter.close();
    }

    public void writeText(City city, double tempDif) throws IOException {
        bufferedWriter.write(String.format("%s%.1f", city.stringExtensionCity(), city.getTemp() + tempDif));
        bufferedWriter.newLine();
    }

    public City getRandomCity() {
        return cities.get(rand.nextInt(cities.size()));
    }

    public double getRandTempDifference() {
        double temp = rand.nextDouble(40) - 20;
        return temp;
    }

}

```

Soubor: functions.php

```php
<?php
    function isLoggedIn() {
        if (isset($_SESSION[session_id()])) {
            return true;
        }
        else {
            return false;
        }
    }

    function logOut() {
        if (isLoggedIn()) {
            unset($_SESSION[session_id()]);
            return true;
        }
        else {
            return false;
        }
    }

    function logIn($username, $password, $db) {
        $password = md5($password);
        $params = array($username, $password);


        try {
            $query = $db->prepare("SELECT COUNT(*) FROM uziv WHERE uziv_login = ? AND uziv_heslo = ?");
            $query->execute($params);
        }
        catch (PDOException $e) {
            die("Database error: " . $e->getMessage());
        }

        if ($query->fetchColumn(0) == 1) {
            session_regenerate_id();
            try {
                $query = $db->prepare("SELECT uziv_id FROM uziv WHERE uziv_login = ? AND uziv_heslo = ?");
                $query->execute($params);
            }
            catch (PDOException $e) {
                die("Database error: " . $e->getMessage());
            }
            $id = $query->fetchColumn(0);
            $_SESSION[session_id()] = $id;
            return true;
        }
        else {
            return false;
        }
    }
?>

```

Soubor: AlgoritmyTest.java

```java
import java.util.*;

public class AlgoritmyTest {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Random rnd = new Random();

        System.out.println("Jak velké pole chcete vygenerovat?");
        int[] pole = new int[sc.nextInt()];
        System.out.println("Pole bude doplněno náhodnými čísly");
        System.out.println("Zadejte nejmenší generovatelné číslo:");
        int min = sc.nextInt();
        System.out.println("Zadejte největší generovatelné číslo:");
        int max = sc.nextInt();

        for (int i = 0; i<pole.length; i++) {
            pole[i] = min + rnd.nextInt(max - min + 1);
        }

        System.out.println();
        prvniVolba();
        int volba = sc.nextInt();

        switch (volba) {
            case 1:
                volbaRazeni();
                volba = sc.nextInt();
                switch (volba) {
                    case 1:
                        vypsaniPole(pole);
                        bubbleSort(pole);
                    break;
                    case 2:
                        vypsaniPole(pole);
                        insertSort(pole);
                    break;
                    case 3:
                        vypsaniPole(pole);
                        selectSort(pole);
                    break;
                    case 4:
                        vypsaniPole(pole);
                        System.out.println();
                        int[] finalpole = quickSort(0, pole.length-1, pole);
                        System.out.println("Seřazené pole:");
                        for (int i = 0; i<finalpole.length; i++) {
                            System.out.format("%6d", finalpole[i]);
                        }
                    break;
                    default:
                        System.out.println("Neplatná volba");
                        System.exit(0);
                    break;
                }
            break;
            case 2:
                System.out.println("Jaký prvek chcete v poli vyhledávat?");
                int x = sc.nextInt();
                volbaVyhledavani();
                volba = sc.nextInt();
                switch (volba) {
                    case 1:
                        vypsaniPole(pole);
                        naivniVyhledavani(pole, x);
                    break;
                    case 2:
                        vypsaniPole(pole);
                        bezZarazky(pole, x);
                    break;
                    case 3:
                        vypsaniPole(pole);
                        seZarazkou(pole, x);
                    break;
```

```
                    case 4:
                        vypsaniPole(pole);
                        int[] finalpole = quickSort(0, pole.length-1, pole);
                        int vysledek = binarniVyhledavani(finalpole, 0, finalpole.length-1, x);
                        if (vysledek == -1) {
                            System.out.println("Prvek se v poli nenachází");
                        } else {
                            System.out.println("Prvek se v poli nachází");
                        }
                    break;
                    default:
                        System.out.println("Neplatná volba");
                        System.exit(0);
                    break;
                }
            break;
            default:
                System.out.println("Neplatná volba");
                System.exit(0);
            break;
        }
    }

    public static void prvniVolba() {
        System.out.println("Jakou operaci chcete provést?");
        System.out.println("1 - Seřadit pole");
        System.out.println("2 - Vyhledat prvek v poli");
    }

    public static void volbaRazeni() {
        System.out.println("Jaký algoritmus chcete použít?");
        System.out.println("1 - Bubble Sort");
        System.out.println("2 - Insert Sort");
        System.out.println("3 - Select Sort");
        System.out.println("4 - Quick Sort");
    }

    public static void volbaVyhledavani() {
        System.out.println("Jaký způsob vyhledávání chcete použít?");
        System.out.println("1 - Naivní");
        System.out.println("2 - Bez zarážky");
        System.out.println("3 - Se zarážkou");
        System.out.println("4 - Binární");
    }

    public static void vypsaniPole (int[] pole) {
        System.out.println("Vygenerované pole:");
        for (int i = 0; i<pole.length; i++) {
            System.out.format("%6d", pole[i]);
        }
        System.out.println();
    }

    public static void bubbleSort(int[] pole) {
        for (int i = 0; i<(pole.length-1); i++) {
            for (int j = 0; j<(pole.length-1-i); j++) {
                if (pole[j] > pole[j+1]) {
                    int pom = pole[j];
                    pole[j] = pole[j+1];
                    pole[j+1] = pom;
                }
            }
        }

        System.out.println("Seřazené pole:");
        for (int i = 0; i<pole.length; i++) {
            System.out.format("%6d", pole[i]);
        }
    }
```

```java
    public static void insertSort(int[] pole) {
        for (int i = 1; i<pole.length; i++) {
            int ins = pole[i];
            int j = i-1;
            while ((j >= 0) && (pole[j] > ins)) {
                pole[j+1] = pole[j];
                j--;
            }
            pole[j+1] = ins;
        }

        System.out.println("Seřazené pole:");
        for (int i = 0; i<pole.length; i++) {
            System.out.format("%6d", pole[i]);
        }
    }

    public static void selectSort(int[] pole) {
        for (int i = 0; i<pole.length-1; i++) {
            int max_pos = pole.length-1-i;
            for (int j = 0; j<pole.length-i; j++) {
                if (pole[j] > pole[max_pos]) {
                    max_pos = j;
                }
            }
            int pom = pole[pole.length-1-i];
            pole[pole.length-1-i] = pole[max_pos];
            pole[max_pos] = pom;
        }

        System.out.println("Seřazené pole:");
        for (int i = 0; i<pole.length; i++) {
            System.out.format("%6d", pole[i]);
        }
    }

    public static int[] quickSort(int l, int r, int[] pole) {
        int i = l;
        int j = r;
        int pivot = pole[((l + r) /2)];

        do {
            while (pole[i] < pivot) i++;
            while (pole[j] > pivot) j--;
            if (i <= j) {
                int pom = pole[i];
                pole[i] = pole[j];
                pole[j] = pom;
                i++;
                j--;
            }
        } while (i < j);

        if ((j - l) > 0) quickSort(l, j, pole);
        if ((r - i) > 0) quickSort(i, r, pole);

        return pole;
    }

    public static void naivniVyhledavani(int[] p, int x) {
        boolean vysledek = false;

        for (int i = 0; i<p.length; i++) {
            if (p[i] == x) {
                vysledek = true;
            }
        }

        if (vysledek == true) {
            System.out.println("Prvek se v poli nachází");
        } else {
            System.out.println("Prvek se v poli nenachází");
        }
    }
```

```java

    public static void bezZarazky(int[] p, int x) {
        boolean vysledek = false;
        int i = 0;

        while (vysledek == false && i < p.length) {
            if (p[i] == x) {
                vysledek = true;
            }
            i++;
        }

        if (vysledek == true) {
            System.out.println("Prvek se v poli nachází");
        } else {
            System.out.println("Prvek se v poli nenachází");
        }
    }

    public static void seZarazkou(int[] p, int x) {
        int[] pole = new int[p.length+1];
        for (int i = 0; i<p.length; i++) {
            pole[i] = p[i];
        }
        pole[p.length] = x;

        boolean vysledek = false;
        int i = 0;

        while (vysledek == false) {
            if (pole[i] == x) {
                vysledek = true;
            }
            i++;
        }

        if (vysledek == true && i < pole.length) {
            System.out.println("Prvek se v poli nachází");
        } else {
            System.out.println("Prvek se v poli nenachází");
        }
    }

    public static int binarniVyhledavani(int[] pole, int l, int r, int x) {
        if (r >= l) {
            int s = l + (r - l) / 2;

            if (pole[s] == x) {
                return s;
            }
            if (pole[s] > x) {
                return binarniVyhledavani(pole, l, s-1, x);
            } else {
                return binarniVyhledavani(pole, s+1, r, x);
            }
        }
        return -1;
    }

}
```

Soubor: BaseConversion.java

```java
import java.util.Scanner;

public class BaseConversion {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter Number: ");
        String cislo = scanner.next();

        System.out.print("From which base: ");
        int fromBase = scanner.nextInt();

        System.out.print("To which base: ");
        int toBase = scanner.nextInt();

        try {
            int decimalValue = Integer.parseInt(cislo, fromBase);

            String result = Integer.toString(decimalValue, toBase);

            System.out.println("Result: " + result.toUpperCase());
        } catch (NumberFormatException e) {
            System.out.println("Invalid number or base");
        }

        scanner.close();
    }
}
```

Soubor: ColorMixerController.java

```java
import java.net.URL;
import java.util.ResourceBundle;

import javafx.beans.value.ChangeListener;
import javafx.beans.value.ObservableValue;
import javafx.fxml.FXML;
import javafx.fxml.Initializable;
import javafx.geometry.Insets;
import javafx.scene.control.Slider;
import javafx.scene.layout.Pane;
import javafx.scene.layout.Background;
import javafx.scene.layout.BackgroundFill;
import javafx.scene.layout.CornerRadii;
import javafx.scene.paint.Color;

public class ColorMixerController implements Initializable {

    //promenee pro pamatovani nastavenych barev
    private int red = 0;
    private int green = 0;
    private int blue = 0;

    //graf. komponenty z FXML
    @FXML
    private Pane pnl_color;
    @FXML
    private Slider sld_red;
    @FXML
    private Slider sld_green;
    @FXML
    private Slider sld_blue;

    @Override
    public void initialize(URL url, ResourceBundle rb) {
        //tato metoda se spusti pri startu aplikace

        //ozivime red slider
        sld_red.valueProperty().addListener(new ChangeListener<Number>() {
            @Override
            public void changed(ObservableValue<? extends Number> observable,
                    Number oldValue, Number newValue) {
                //sem vlozit kod, ktery se spusti pri pohnuti se sliderem
                //stahni novou hodnotu cerveneho slideru a uloz ji do promenne
                red = newValue.intValue();
                //prebarvi pozadi panelu
                setPanelBackGround();
            }
        });

        //ozivime green slider
        sld_green.valueProperty().addListener(new ChangeListener<Number>() {
            @Override
            public void changed(ObservableValue<? extends Number> observable,
                    Number oldValue, Number newValue) {
                //sem vlozit kod, ktery se spusti pri pohnuti se sliderem
                //stahni novou hodnotu cerveneho slideru a uloz ji do promenne
                green = newValue.intValue();
                //prebarvi pozadi panelu
                setPanelBackGround();
            }
        });
```

```
        //ozivime blue slider
        sld_blue.valueProperty().addListener(new ChangeListener<Number>() {
            @Override
            public void changed(ObservableValue<? extends Number> observable,
                    Number oldValue, Number newValue) {
                //sem vlozit kod, ktery se spusti pri pohnuti se sliderem
                //stahni novou hodnotu cerveneho slideru a uloz ji do promenne
                blue = newValue.intValue();
                //prebarvi pozadi panelu
                setPanelBackGround();
            }
        });

        //spustime setBackGround - kvuli nastaveni prvni cerne barvy
        setPanelBackGround();
    }

    private void setPanelBackGround() {
        pnl_color.setBackground(new Background(
                new BackgroundFill(Color.rgb(red, green, blue),
                        CornerRadii.EMPTY, Insets.EMPTY)));
    }
}
```

Soubor: ColorMixerApplication.java

```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.AnchorPane;
import javafx.fxml.FXMLLoader;


public class ColorMixerApplication extends Application {
    @Override
    public void start(Stage primaryStage) {
        try {
            AnchorPane root = (AnchorPane)FXMLLoader.load(getClass().getResource("ColorMixerV2.fxml"));
            Scene scene = new Scene(root);
            primaryStage.setScene(scene);
            primaryStage.show();
        } catch(Exception e) {
            e.printStackTrace();
        }
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```